LasForecast: Lasso for Predictive Regressions

Table of contents

1 Introduction

`LasForecast` provides a unified framework for economic forecasting using penalized linear predictive regression. The package implements LASSO-family estimators specifically designed

for predictive regression with potentially persistent regressors, a setting common in macroeconomics and finance. It automates the following steps: **parameter tuning**, **model estimation**, **rolling/expanding window backtesting**, and **visualization**.

The package primarily implements the methods from three papers:

- Lee et al. (2022) "On LASSO for Predictive Regression", *Journal of Econometrics*, 229(2), 322–349.
- Mei and Shi (2024) "On LASSO for High Dimensional Predictive Regression", *Journal of Econometrics*, 242(2), 105809.
- Gao et al. (2026) "LASSO Inference for High Dimensional Predictive Regressions", *Journal of Econometrics*, 255, 106240.

1.1 Model

Consider the canonical linear predictive regression

$$y_t = \alpha^* + W_{t-1}^\top \theta^* + u_t, \quad t = 1, \dots, n, \tag{1}$$

where y_t is the target variable (e.g., stock excess return or inflation), $W_t = (X_t^\top, Z_t^\top)^\top$ is a p-dimensional vector of predictors observed at time t, $\theta^* = (\beta^{*\top}, \gamma^{*\top})^\top \in \mathbb{R}^p$ is the true coefficient vector, and u_t is the prediction error. The intercept α^* is unpenalized in all LASSO formulations below.

The defining feature of predictive regressions is that the regressors W_t carry heterogeneous degrees of persistence. We distinguish two types:

Stationary regressors. The $p_z \times 1$ vector $Z_t = (z_{1,t}, \dots, z_{p_z,t})^\top$ collects short-memory, $I(0)$ predictors. In financial applications, these include variables such as stock variance, inflation, and net equity expansion.

Unit root regressors. The $p_x \times 1$ vector $X_t = (x_{1,t}, \dots, x_{p_x,t})^\top$ collects persistent predictors, each following a unit root process

$$x_{j,t} = x_{j,t-1} + e_{j,t}, \quad j = 1, \dots, p_x, \tag{2}$$

where $e_{j,t}$ is the innovation. Common persistent predictors include the dividend-price ratio, earnings-price ratio, and bond term spread. This pure unit root specification is the baseline model studied by Lee et al. (2022) and Mei and Shi (2024).

The challenge of mixed roots. The coexistence of stationary and persistent regressors creates fundamental difficulties for penalized estimation:

(i) *Different convergence rates.* OLS estimators of coefficients on stationary regressors converge at the $\sqrt{n}$ rate, whereas those on unit root regressors converge at the n rate. A uniform penalty cannot simultaneously accommodate both rates.

(ii) *Stambaugh bias.* The correlation between the regressor innovation e_t and the prediction error u_t introduces an endogeneity bias in finite samples that grows with persistence (Stambaugh 1999).

In practice, the econometrician typically does not know which regressors are stationary and which are persistent; nor is the cointegration structure known. Writing $p = p_x + p_z$ for the total number of regressors, we consider two asymptotic frameworks:

- *Low-dimensional:* p is fixed and $n \to \infty$ (Lee et al. 2022).
- *High-dimensional:* $p = p(n) \to \infty$, possibly $p > n$, while the true model is sparse with $s = |\{j : \theta_j^* \neq 0\}|$ nonzero coefficients (Mei and Shi 2024; Gao et al. 2026).

Throughout, the innovations (e_t, u_t) are allowed to be correlated contemporaneously.

1.2 LASSO Variants

Let $W = (W_0, W_1, \dots, W_{n-1})^\top$ denote the $n \times p$ predictor matrix and $y = (y_1, \dots, y_n)^\top$ the response vector. For notational economy, the intercept α^* is absorbed through demeaning.

Plain LASSO (PLasso). The original Tibshirani (1996) estimator solves

$$\hat{\theta}^P = \operatorname*{argmin}_\theta \left\{ \frac{1}{n} \|y - W\theta\|_2^2 + \lambda \|\theta\|_1 \right\}, \tag{3}$$

where $\lambda > 0$ is a tuning parameter and $\|\theta\|_1 = \sum_{j=1}^p |\theta_j|$. PLasso applies a uniform penalty to all coefficients regardless of the scale or persistence of the corresponding regressor. Consequently, it is *not* scale-invariant. In the presence of mixed roots, persistent regressors have column norms $\|w_j\|_2 = O_p(n)$ while stationary regressors have $\|w_j\|_2 = O_p(\sqrt{n})$, so a uniform penalty under-shrinks the former and over-shrinks the latter.

Standardized LASSO (SLasso). To address scale dependence, a widely adopted practice, and the default in the R package `glmnet`, is to standardize each regressor by its sample standard deviation. Define $\hat{\sigma}_j = \sqrt{n^{-1} \sum_{t=1}^n (w_{j,t-1} - \bar{w}_j)^2}$ and the diagonal matrix $D = \operatorname{diag}(\hat{\sigma}_1, \dots, \hat{\sigma}_p)$. The SLasso estimator is

$$\hat{\theta}^S = \operatorname*{argmin}_\theta \left\{ \frac{1}{n} \|y - W\theta\|_2^2 + \lambda \|D\theta\|_1 \right\}. \tag{4}$$

Since $\hat{\sigma}_j$ absorbs the regressor's marginal variation, the effective penalty $\lambda \hat{\sigma}_j |\theta_j|$ on each coefficient is commensurate across regressor types. Mei and Shi (2024) show that this scale-invariance property is crucial: it aligns the magnitudes of persistent and stationary components, maintaining LASSO's *estimation* consistency under mixed roots.

Adaptive LASSO (ALasso). The Adaptive LASSO of Zou (2006) introduces data-driven penalty weights to achieve the oracle property. For the predictive regression in Equation 1, it solves (note: the $1/n$ factor used in PLasso/SLasso is absorbed into λ_n here)

$$\hat{\theta}^A = \underset{\theta}{\operatorname{argmin}} \left\{ \|y - W\theta\|_2^2 + \lambda_n \sum_{j=1}^{p} \hat{\tau}_j |\theta_j| \right\}, \tag{5}$$

where $\hat{\tau}_j = |\hat{\theta}_j^{\text{init}}|^{-\gamma}$ for some initial consistent estimator $\hat{\theta}^{\text{init}}$ (typically OLS when $p < n$ and Lasso when $p > n$), and $\gamma > 0$ controls the aggressiveness of the penalty. When $\hat{\theta}_j^{\text{init}}$ is close to zero (an inactive predictor), the weight $\hat{\tau}_j$ is large and the penalty effectively forces $\hat{\theta}_j^A$ to zero. Conversely, when $\hat{\theta}_j^{\text{init}}$ is far from zero (an active predictor), the penalty is small and the estimate approaches the OLS value.

Lee et al. (2022) show that ALasso attains the oracle asymptotic distribution for the active set in predictive regressions with mixed-persistence regressors. However, in the presence of cointegrated unit root regressors, ALasso only *partially* succeeds:

- When $p < n$, ALasso can use OLS as a consistent initial estimator. While it preserves the cointegration rank, it may fail to exclude all inactive cointegrating variables, potentially leading to over-selection. This limitation motivates the development of TALasso (see Equation 6).
- When $p > n$, no consistent initial estimator is available if cointegration is present, as of the development of this package. Mei and Shi (2024) show that SLasso tends to over-penalize coefficients of cointegrated predictors in this setting. Without a consistent initial estimator, both ALasso and TALasso fail to work in high dimension.

Twin Adaptive LASSO (TALasso). Lee et al. (2022) propose the Twin Adaptive LASSO, which applies ALasso twice to handle cointegration. In the first round, ALasso is run as in Equation 5 to obtain the estimated active set $\hat{M}^A = \{j : \hat{\theta}_j^A \neq 0\}$. In the second round, post-selection OLS is performed on $\hat{M}^A$ to obtain refined estimates $\hat{\theta}_j^{\text{po}}$, which serve as new penalty weights $\hat{\tau}_j^{\text{po}} = |\hat{\theta}_j^{\text{po}}|^{-\gamma}$. The TALasso estimator is

$$\hat{\theta}_{\hat{M}^A}^{TA} = \underset{\theta}{\operatorname{argmin}} \left\{ \left\| y - \sum_{j \in \hat{M}^A} w_j \theta_j \right\|_2^2 + \lambda_n \sum_{j \in \hat{M}^A} \hat{\tau}_j^{\text{po}} |\theta_j| \right\}, \tag{6}$$

with $\hat{\theta}_j^{TA} = 0$ for $j \notin \hat{M}^A$. The key insight is that, after the first-round ALasso selects $\hat{M}^A$, the over-selected cointegrating variables lose their cointegrating ties in the post-selection regression. Their post-selection OLS estimates converge at the n rate, producing penalty weights of order n^γ, heavy enough to eliminate them. TALasso is the first estimator to achieve the oracle property in predictive regressions *without* prior knowledge of which regressors are stationary, persistent, or cointegrated.

IVX-Desparsified LASSO (XDLasso). The XDLasso of Gao et al. (2026) is designed for *inference* rather than prediction or variable selection. It combines three ingredients: the SLasso estimator, the IVX instrument of Phillips and Magdalinos (2009), and the desparsification idea of Zhang and Zhang (2014). The procedure targets a specific coefficient θ_j^*:

1. Obtain the SLasso estimator $\widehat{\theta}^S$ from Equation 4 and save the residual $\widehat{u}_t = y_t - W_{t-1}^\top \widehat{\theta}^S$.
2. Construct the IVX instrument $\zeta_{j,t} = \sum_{s=1}^{t} \rho_\zeta^{t-s} \Delta w_{j,s}$, where $\rho_\zeta = 1 - C_\zeta / n^\tau$ for a user-chosen $\tau \in (0,1)$, and standardize it as $\widetilde{\zeta}_{j,t} = \zeta_{j,t} / \widehat{\gamma}_j$ with $\widehat{\gamma}_j$ the sample standard deviation of $\zeta_{j,t}$.
3. Run an auxiliary LASSO regression of $\widetilde{\zeta}_{j,t-1}$ on $W_{-j,t-1}$ (all regressors except w_j) to obtain the residual score vector $\widehat{r}_{j,t} = \widetilde{\zeta}_{j,t} - W_{-j,t}^\top \widehat{\varphi}^{(j)}$.
4. Compute the bias-corrected estimator and its standard error (see Equation 7 below).
5. Compute $t_j^{XD} = (\widehat{\theta}_j^{XD} - \theta_{0,j}) / \widehat{\omega}_j^{XD}$ and compare with the standard normal critical value.

The bias-corrected estimator and standard error in Step 4 are

$$\widehat{\theta}_j^{XD} = \widehat{\theta}_j^S + \frac{\sum_{t=1}^n \widehat{r}_{j,t-1}\, \widehat{u}_t}{\sum_{t=1}^n \widehat{r}_{j,t-1}\, w_{j,t-1}}, \qquad \widehat{\omega}_j^{XD} = \frac{\widehat{\sigma}_u \sqrt{\sum_{t=1}^n \widehat{r}_{j,t-1}^2}}{\left| \sum_{t=1}^n \widehat{r}_{j,t-1}\, w_{j,t-1} \right|}, \tag{7}$$

where $\widehat{\sigma}_u^2 = n^{-1} \sum_{t=1}^n \widehat{u}_t^2$.

1.3 Summary and Comparison

The table below summarizes the properties of the five LASSO estimators across key dimensions.

Table 1: Comparison of LASSO estimators for predictive regressions.

	PLasso	SLasso	ALasso	TALasso	XDLasso
Dimension	High	High	High	Low	High
Mixed roots	No	I(0) + I(1)	I(0) + I(1)	I(0) + I(1) + cointegration	I(0) + I(1)
Variable selection	No	Partial	Yes	Yes	N/A
Valid inference	N/A	N/A	N/A	N/A	Yes

Several practical guidelines emerge:

(i) For *prediction* with a large number of mixed-root predictors, SLasso is the recommended estimator, as it maintains consistency across regressor types and is computationally efficient.

(ii) For *inference* on individual coefficients in high-dimensional predictive regressions, XDLasso delivers valid t-tests and Wald tests regardless of regressor persistence, provided the model is sparse.

(iii) For *variable selection* with mixed $I(0)$ and $I(1)$ regressors, ALasso achieves oracle consistency. When cointegration among unit root regressors is potentially present, TALasso restores oracle consistency in the low-dimensional setting (p fixed). Variable selection under mixed roots with cointegration in high dimensions ($p > n$) remains an open problem.

2 Real Data Illustration

2.1 Data

The package ships with the FRED-MD dataset, a panel of macroeconomic time series maintained by the Federal Reserve Bank of St. Louis (McCracken and Ng 2016). Two versions are included: `fredmd` (transformed to stationarity following McCracken and Ng) and `fredmd` (untransformed levels). Both contain 785 monthly observations (December 1959 – April 2025) across 112 series.

```
data("fredmd")
# fredmd <- fredmd[fredmd$date >= as.Date("1979-12-01") &
#   fredmd$date <= as.Date("2019-12-01"), ]
dim(fredmd)
```

```
[1] 785 113
```

```
head(fredmd[, 1:6])
```

```
# A tibble: 6 x 6
  date       infl_cpi UNRATE     RPI  W875RX1 DPCERA3M086SBEA
  <date>        <dbl>  <dbl>   <dbl>    <dbl>           <dbl>
1 1959-12-01    0.204    5.3 0.0102  0.0114          -0.00114
2 1960-01-01   -0.136    5.2 0.00323 0.00466          0.00279
3 1960-02-01    0.136    4.8 0.00115 0.000906         0.00436
4 1960-03-01    0        5.4 0.00188 0.000905         0.0141
5 1960-04-01    0.441    5.2 0.00346 0.00361          0.0153
6 1960-05-01    0.102    5.1 0.00241 0.00243         -0.0204
```

The full sample contains 785 monthly observations (December 1959 – April 2025). The target variable is `infl_cpi` (CPI inflation rate). The remaining 111 columns serve as predictors.

2.2 Estimation

The `lasso()` function is the unified interface for fitting all LASSO variants. It accepts flexible inputs: when `x` is a data frame, the function auto-detects and removes the date column, and when `y` is specified as a column name string, it extracts `y` from `x` and uses all remaining columns as predictors. Internally, `lasso()` calls `prepare_input()` for input processing, `train_lasso()` for tuning parameter selection, and the appropriate estimation routine.

2.2.1 PLasso, SLasso, ALasso, TALasso

We use SLasso as a running example to illustrate the estimation workflow. We hold out the final 12 months so that the forecasts at the end of this subsection are genuinely out-of-sample. SLasso is fit via `method = "lasso"` with `scale_x = TRUE` (the default):

```
# Hold out the last 12 months: mod_s never sees them during fitting, so the
# forecasts below are genuinely out-of-sample rather than in-sample fits.
mod_s <- lasso(fredmd[1:(nrow(fredmd) - 12), ],
  y = "infl_cpi", method = "lasso",
  scale_x = TRUE, train_method = "cv"
)
```

The returned object is of class `lasforecast_model`. Key components include the selected penalty parameter `lambda` and the coefficient vector `coefficients`:

```
mod_s$lambda
```

```
[1] 0.01016581
```

```
mod_s$method
```

```
[1] "lasso"
```

The `coef()` method returns the full coefficient vector (intercept first, then slopes). Nonzero entries correspond to selected predictors:

```r
beta <- coef(mod_s)
names(beta) <- c(
  "(Intercept)",
  setdiff(names(fredmd), c("date", "infl_cpi"))
)
beta[beta != 0]
```

```
  (Intercept)        UNRATE       W875RX1       RETAILx        IPNMAT
  2.636378919   0.006875706 -2.179056368   0.051365917 -0.602485176
    UEMP5TO14 CES1021000001      USWTRADE        USGOVT        AWHMAN
  0.088703136   0.240957606   0.212050873 -1.557810128 -0.066498965
     HOUSTNE        HOUSTW       AMDMUOx       BUSINVx      ISRATIOx
 -0.007637499   0.061815316   0.924298889 11.683645113 -1.453851798
      M2REAL      TOTRESNS      BUSLOANS      NONREVSL        CONSPI
 -7.577389816  -0.243363935  -0.354212002 -1.194222671 -0.003027575
      S&P 500 S&P div yield           GS5      TB3SMFFM      TB6SMFFM
  0.062277963  -0.111592921   0.144405053 -0.037460812 -0.045439409
      T5YFFM        AAAFFM       EXSZUSx     OILPRICEx      CPITRNSL
 -0.015522882  -0.013120684  -0.503648093   0.203226880   3.226424574
  CUSR0000SAC   CUSR0000SAD CES2000000008   DTCOLNVHFNM
  0.205004788   0.895142257   0.133875520   0.021164609
```

Time alignment. The predictive regression model Equation 1 pairs y_t with W_{t-1}, so the response is one period ahead of the predictors. Both `lasso()` and `xdlasso()` handle this alignment automatically via the `predictive` argument. When the input is a data.frame with a detected date column (as with `fredmd`), `predictive` defaults to `TRUE` and the function internally shifts the data so that y_t is regressed on x_{t-1}, dropping one observation. When the input is a raw matrix (no date information), `predictive` defaults to `FALSE`, assuming the user has pre-aligned the data. The user can always override with an explicit `predictive = TRUE` or `FALSE`.

The `predict()` method computes fitted values for new predictor data via the `newx` argument. Since the model is a predictive regression, to forecast inflation in the held-out last 12 months, we feed predictors from the *preceding* 12 months. Because `mod_s` was fit without these observations, the forecasts are genuinely out-of-sample:

```r
n <- nrow(fredmd)
target_rows <- (n - 11):n
pred_rows <- target_rows - 1
x_new <- as.matrix(fredmd[pred_rows, !(names(fredmd) %in% c("date", "infl_cpi"))])
y_hat <- predict(mod_s, newx = x_new)
data.frame(
```

```
  date = fredmd$date[target_rows],
  actual = fredmd$infl_cpi[target_rows],
  predicted = y_hat
)
```

```
         date       actual predicted
1  2024-05-01  0.039606744 0.3964391
2  2024-06-01 -0.002874155 0.2096250
3  2024-07-01  0.138823090 0.2896801
4  2024-08-01  0.180023213 0.3986106
5  2024-09-01  0.228941469 0.2662277
6  2024-10-01  0.226200298 0.3319717
7  2024-11-01  0.280057714 0.4246771
8  2024-12-01  0.364008401 0.3435789
9  2025-01-01  0.465848376 0.3324927
10 2025-02-01  0.215696456 0.3444534
11 2025-03-01 -0.050047703 0.2075964
12 2025-04-01  0.220647154 0.1238810
```

PLasso. The plain LASSO without standardization. Set `scale_x = FALSE`:

```
mod_p <- lasso(fredmd,
  y = "infl_cpi", method = "lasso",
  scale_x = FALSE, train_method = "cv"
)
```

ALasso. The Adaptive LASSO uses data-driven penalty weights from an initial estimator. The `model` argument controls the initial estimator: `"ols"` (when $p < n$), `"lasso"`, or `"ridge"`. If `model = NULL` (default), the function auto-selects based on dimensionality.

```
mod_a <- lasso(fredmd, y = "infl_cpi", method = "alasso", train_method = "cv")
```

TALasso. The Twin Adaptive LASSO runs ALasso twice: the first round selects variables, and the second round refines with updated penalty weights from post-selection OLS. As in Lee et al. (2022), the TALasso is intened to deal with cointegrated regressor in low dimension. A warning will be released if a shrinkage initial estiamtor is invoked.

```
mod_ta <- lasso(fredmd, y = "infl_cpi", method = "talasso", train_method = "cv")
```

```
Warning in lasso(fredmd, y = "infl_cpi", method = "talasso", train_method =
"cv"): TALasso is designed for low-dimensional predictive regressions, but the
first-stage adaptive Lasso used a shrinkage initial estimator ('lasso', chosen
when p > 2 * sqrt(n) or set via model). Consider method = 'alasso' for
high-dimensional data.
```

```r
mod_ta <- lasso(fredmd[, 1:30], y = "infl_cpi", method = "talasso", train_method = "cv")
```

Post-Lasso OLS. Each method has a post-selection variant (`"post_lasso"`, `"post_alasso"`, `"post_talasso"`) that uses the corresponding LASSO for variable selection, then re-estimates by OLS on the selected set to reduce shrinkage bias.

```r
mod_post <- lasso(fredmd, y = "infl_cpi", method = "post_alasso", train_method = "cv")
```

2.2.2 XDLasso

The methods above (SLasso, PLasso, ALasso, TALasso) produce point estimates and forecasts, but do not deliver valid inference (confidence intervals, p-values) for individual coefficients. `xdlasso()` fills this gap by implementing the IVX-desparsified LASSO of Gao et al. (2026), which provides asymptotically normal t-statistics regardless of whether the regressor of interest is stationary or persistent.

We illustrate with a *predictive Phillips curve*: forecasting CPI inflation from the unemployment rate (`UNRATE`) while controlling for 110 other macroeconomic predictors. This is the empirical application studied in Gao et al. (2026).

Like `lasso()`, `xdlasso()` accepts data.frame, tibble, or tsibble inputs with `y` specified as a column name. The focal predictors for inference are specified by name via the `d` argument (or by column index via `d_ind`).

```r
fit_xd <- xdlasso(fredmd,
  y = "infl_cpi",
  d = c("UNRATE", "S&P 500"),
  train_method = "cv", joint_test = TRUE
)
```

The `summary()` method prints a coefficient table for the focal variables, debiased IVX estimates with standard errors, t-values, and p-values, then the Wald test (if requested), followed by the Lasso-selected control variables from the first stage.

```r
summary(fit_xd)
```

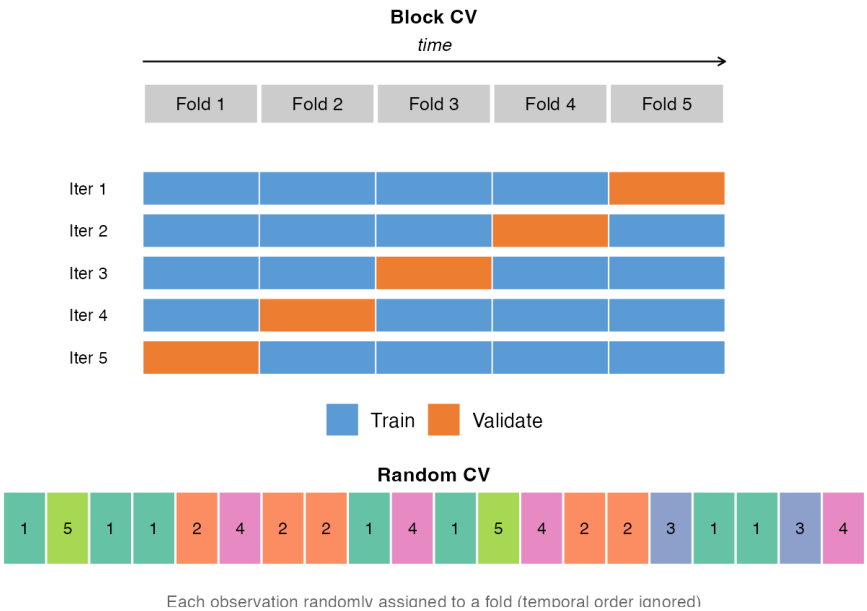
```
IVX-Desparsified Lasso Inference
---
Focal coefficients:
         Estimate Std. Error t value Pr(>|t|)
UNRATE  -0.024175   0.077511 -0.3119   0.7551
S&P 500  0.862781   0.545517  1.5816   0.1137

Wald test: chi2(2) = 2.605, p = 0.2718

Lasso-selected controls (first-step estimates):
      W875RX1       RETAILx         IPNMAT     UEMP5TO14 CES1021000001
    -1.707175      0.138280     -0.308007      0.054449      0.162359
       USGOVT         AWHMAN         HOUSTW       AMDMUOx       BUSINVx
    -1.326736     -0.064453      0.047981      0.988712     11.106750
     ISRATIOx         M2REAL      TOTRESNS      BUSLOANS      NONREVSL
    -1.211713     -7.118165     -0.217056     -0.251719     -1.037176
       CONSPI S&P div yield           GS5      TB3SMFFM      TB6SMFFM
    -0.001387     -0.073438      0.137000     -0.045164     -0.047714
       T5YFFM         AAAFFM       EXSZUSx     OILPRICEx      CPITRNSL
    -0.009205     -0.012545     -0.407688      0.198425      3.399647
  CUSR0000SAD
     0.120331
```

2.2.3 Tuning Parameter Selection

The penalty parameter λ is selected by the `train_method` argument. The package offers three families of tuning methods, each with different assumptions about the data.

Information criteria (`"aic"`, `"bic"`, `"aicc"`, `"hqc"`). These fit the model once along the full λ path and select the λ minimizing the chosen criterion. BIC tends to select sparser models than AIC. These methods are fast and do not require sample splitting. HQC (Hannan–Quinn) lies between AIC and BIC in sparsity.

Block cross-validation (`"cv"`, `"cv_random"`). The `"cv"` method splits the sample into k contiguous blocks and uses each block as a validation fold. This preserves the temporal ordering within each block but allows temporal gaps at fold boundaries. The `"cv_random"` method assigns observations to folds randomly, ignoring temporal structure; it is in general inappropriate for dependent data.



Time-series cross-validation (`"timeslice"`). This method respects the temporal ordering strictly: training always uses past data and validation uses future data. Key parameters:

- `initial_window`: size of the first training set (default: 70% of n)
- `horizon`: number of periods in each validation set (default: 1)
- `fixed_window`: if `TRUE` (default), training window rolls forward at fixed size; if `FALSE`, it expands
- `skip`: number of periods to skip between successive splits (default: 0)

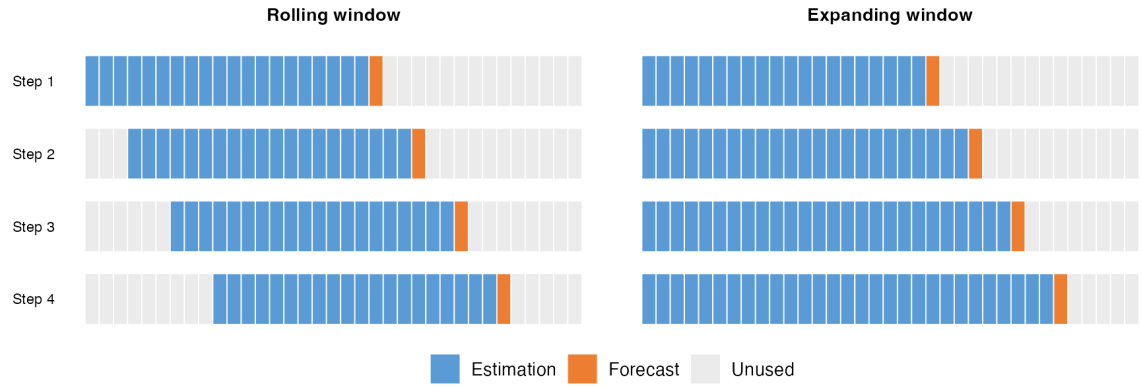

```
mod_ts <- lasso(fredmd,
  y = "infl_cpi", method = "lasso",
  train_method = "timeslice",
```

```
  initial_window = 240, horizon = 1, fixed_window = TRUE, skip = 0
)
print(c(mod_ts$lambda, mod_s$lambda))
```

```
[1] 0.01470193 0.01016581
```

2.3 Backtesting

The `backtest()` function evaluates and compares forecasting methods by repeatedly estimating models on historical data and scoring their out-of-sample predictions. At each step, the model is trained on the `roll_window` most recent observations and used to generate a one-step-ahead (or h-step-ahead) forecast. The window then advances one period and the process repeats, producing a time series of pseudo out-of-sample forecasts for every method specified.



Within each window, the predictive time alignment $y_t \sim W_{t-1}$ is maintained automatically. The function computes the requested loss metrics (`rmse`, `mae`, `mse`, `mape`, `r2oos`) across all out-of-sample forecasts, and optionally reports them as ratios relative to a benchmark method (default: Random Walk with Drift).

Alongside the penalized estimators, `backtest()` and `roll_predict()` accept the standard benchmarks `"RW"`, `"RWwD"`, `"OLS"`, and `"ARMA"`. The `"ARMA"` method is a univariate ARMA model on the target: its order can be fixed via `arma_order = c(p, d, q)`, or left unspecified, in which case the AR and MA orders are selected automatically by information criterion within each window.

```
bt_results <- backtest(
  x = fredmd,
  y = "infl_cpi",
  methods = c("PLasso", "SLasso", "ALasso"),
  roll_window = 360,
```

```
  h = 1,
  loss = c("rmse", "mae"),
  benchmark = "RWwD",
  verbose = FALSE,
  train_method = "cv"
)
```

```
summary(bt_results)
```

```
  Method      RMSE       MAE RMSE_Ratio MAE_Ratio
1   RWwD 0.3042642 0.2218854  1.0000000 1.0000000
2 PLasso 0.2970539 0.1930847  0.9763027 0.8702000
3 SLasso 0.2460384 0.1767522  0.8086342 0.7965924
4 ALasso 0.2651720 0.1875667  0.8715189 0.8453314
```

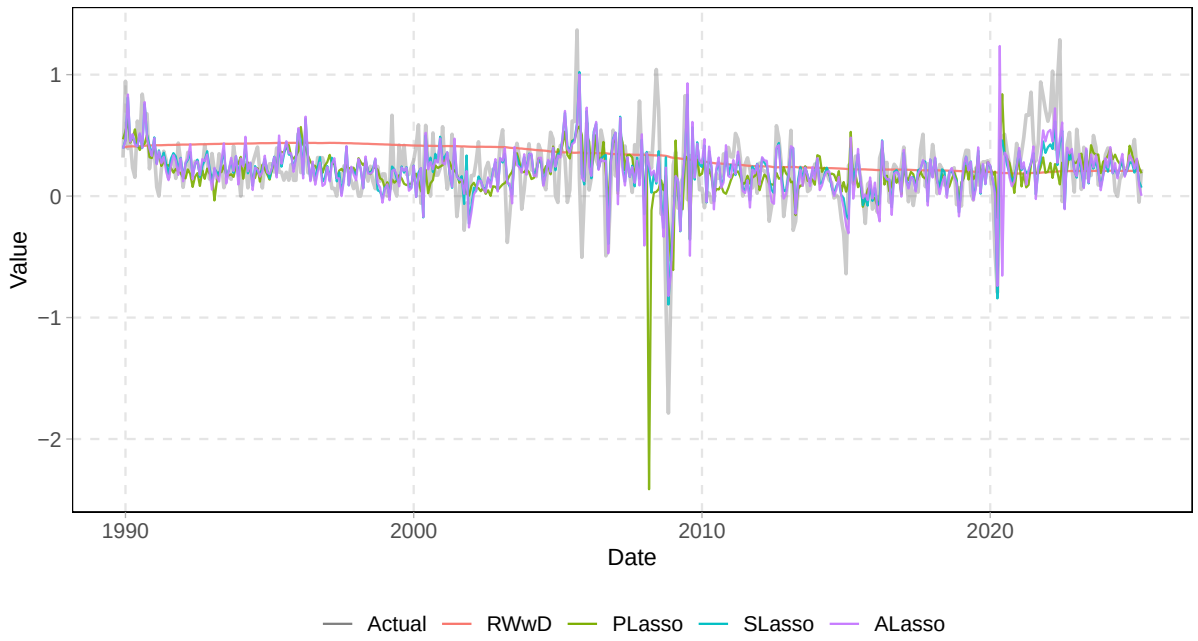
2.4 Visualization

The `autoplot()` method for backtest objects supports multiple plot types.

2.4.1 Time Series Plot: Actual v.s. Predicted

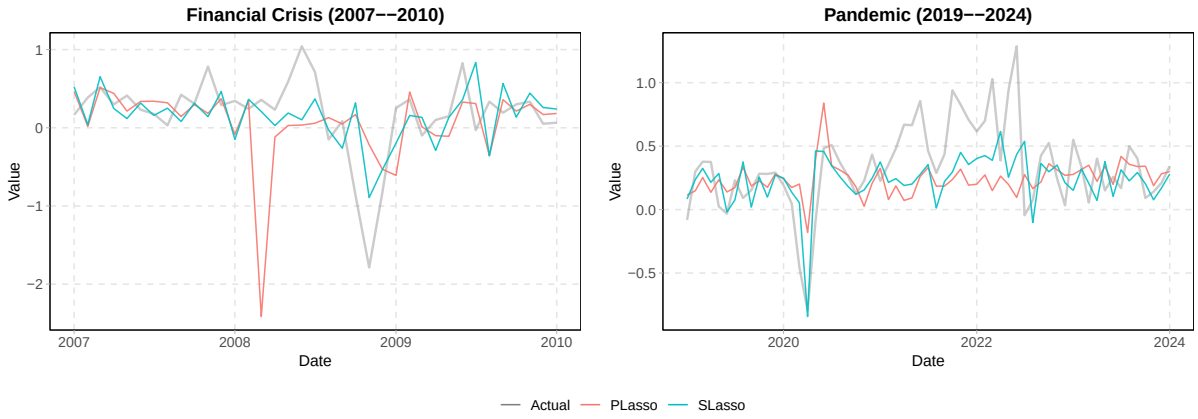
The `"forecasts"` type overlays the actual target series (gray) with predictions from each method:

```
autoplot(bt_results, type = "forecasts")
```



Use `methods_to_plot` to select specific methods and `date_range` to zoom into a time window: zooming into two turbulent episodes, the 2007–09 financial crisis and the 2020–21 pandemic, reveals how well the Lasso forecasts track actual inflation during extreme swings:

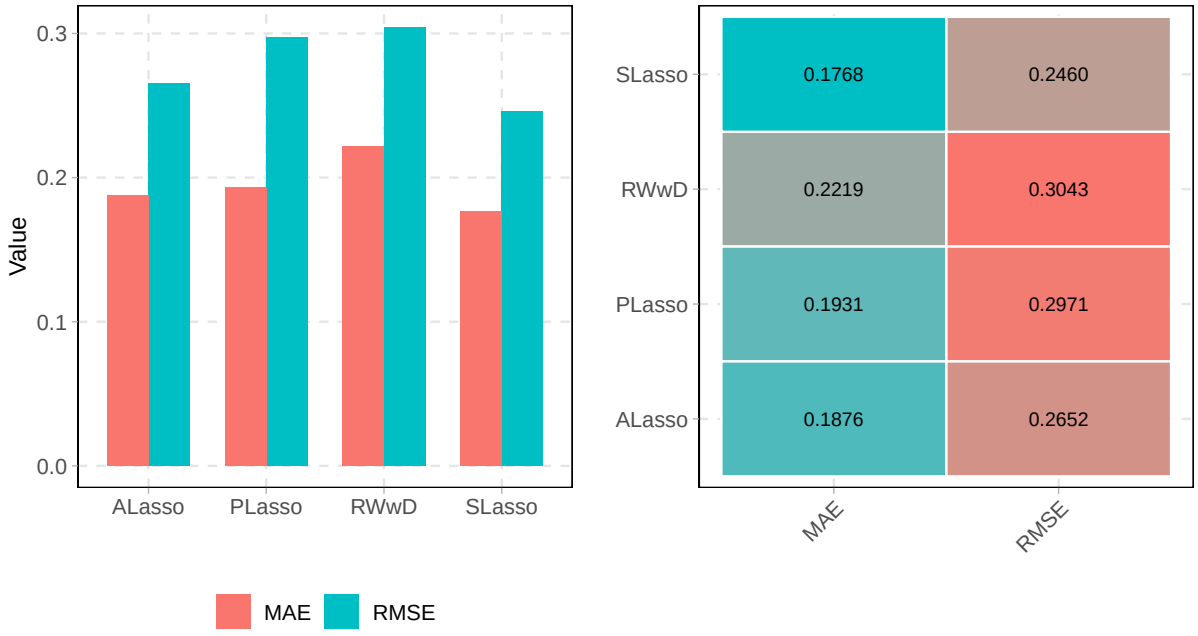
```
library(patchwork)
p_crisis <- autoplot(bt_results,
  type = "forecasts",
  methods_to_plot = c("PLasso", "SLasso"),
  date_range = c("2007-01-01", "2010-01-01")
) + ggtitle("Financial Crisis (2007--2010)")

p_pandemic <- autoplot(bt_results,
  type = "forecasts",
  methods_to_plot = c("PLasso", "SLasso"),
  date_range = c("2019-01-01", "2024-01-01")
) + ggtitle("Pandemic (2019--2024)")

p_crisis + p_pandemic + plot_layout(guides = "collect") &
  theme(legend.position = "bottom")
```



2.4.2 Accuracy Metrics

The `"bar"` and `"heatmap"` plot types visualize loss metrics across methods. Set `ratio = TRUE` to show values relative to the benchmark:

```
autoplot(bt_results, type = "bar") + autoplot(bt_results, type = "heatmap")
```

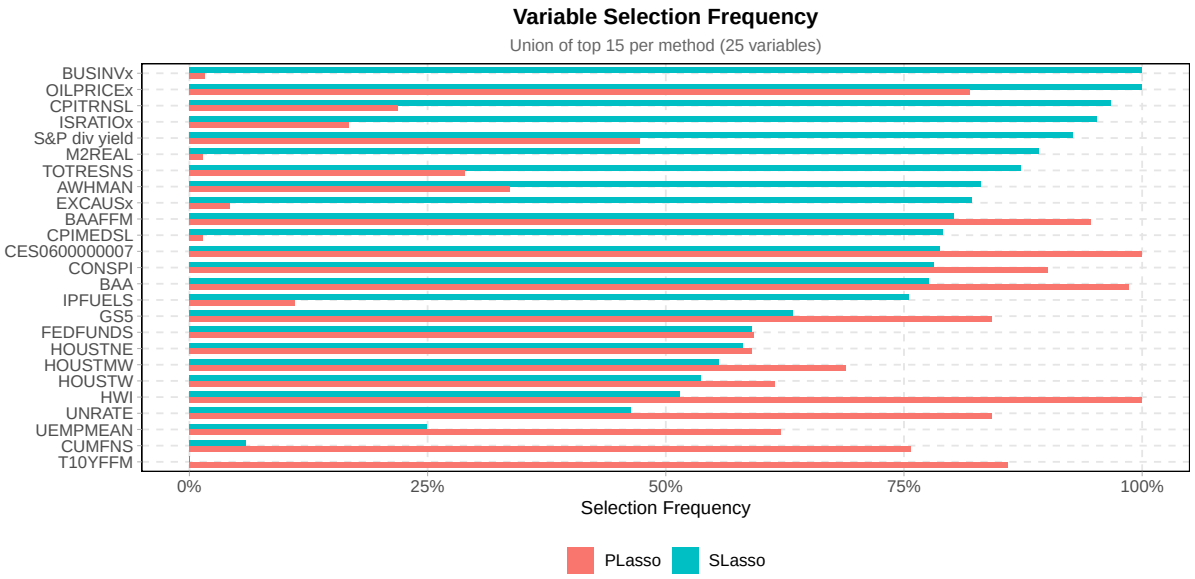


2.4.3 Variable Selection

Lasso methods perform implicit variable selection at every rolling window. The `"frequency"` type shows how often each predictor is selected (nonzero coefficient) across all windows. The

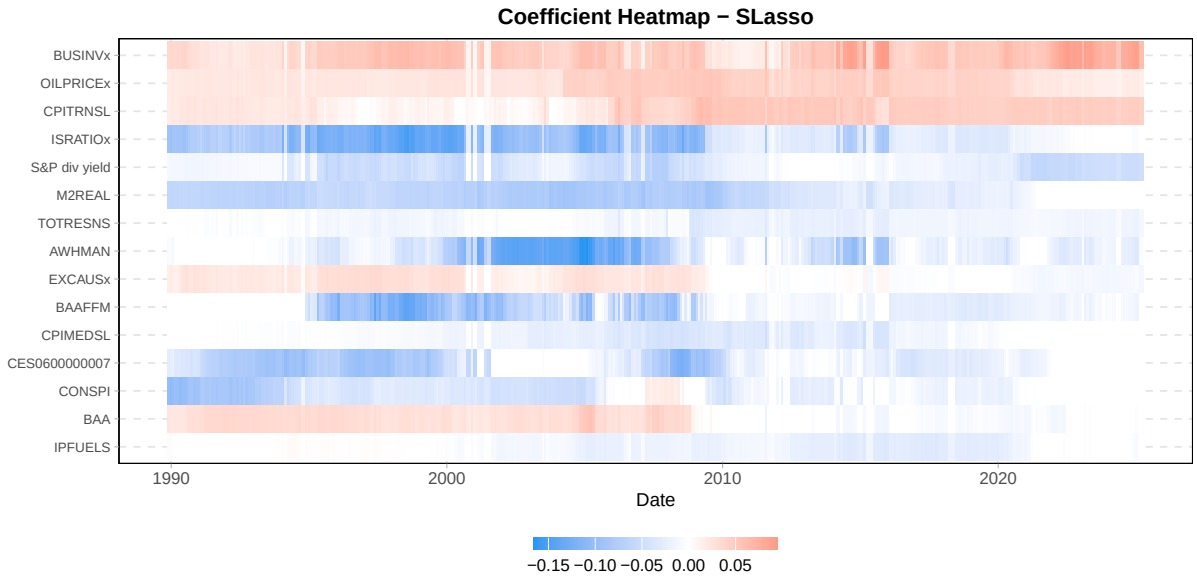
`methods` argument selects which methods to display; variables are sorted by the frequency of the first method listed. The displayed set is the union of each method's top `top_n` variables:

```
autoplot(bt_results, type = "frequency", methods = c("SLasso", "PLasso"), top_n = 15)
```



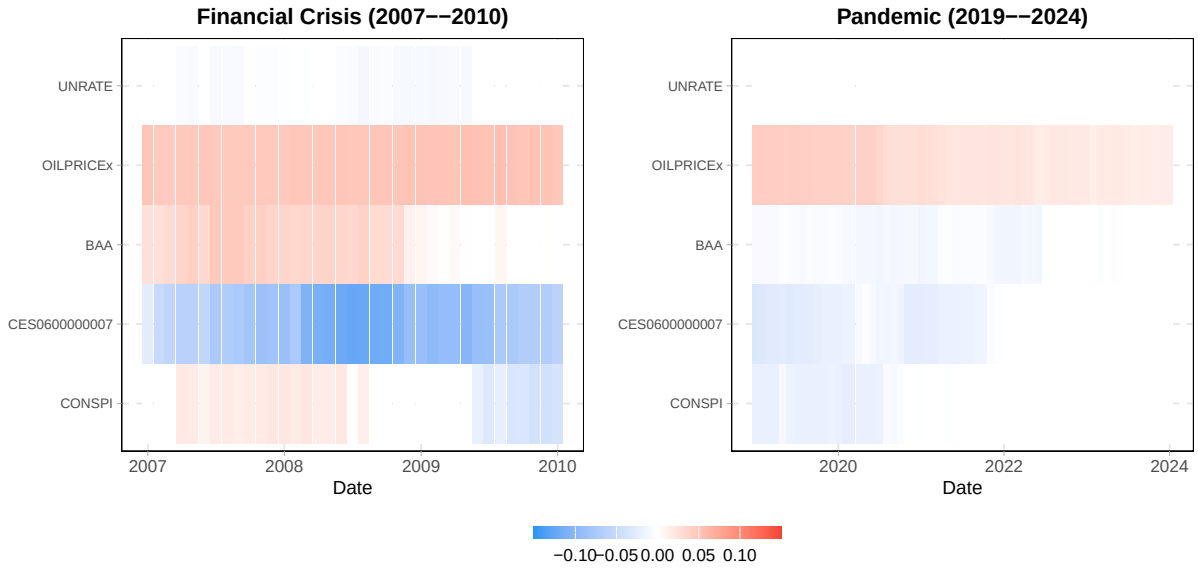
The `"coef_heatmap"` type reveals the time-varying sparsity pattern, showing which predictors enter or leave the model as the estimation window rolls forward. By default, coefficients are *standardized*: each $\hat{\beta}_j$ is multiplied by the sample standard deviation $\hat{\sigma}_j$ of the corresponding predictor, so the color intensity represents the effect per one-standard-deviation change.

```
autoplot(bt_results, type = "coef_heatmap", methods = "SLasso", top_n = 15)
```



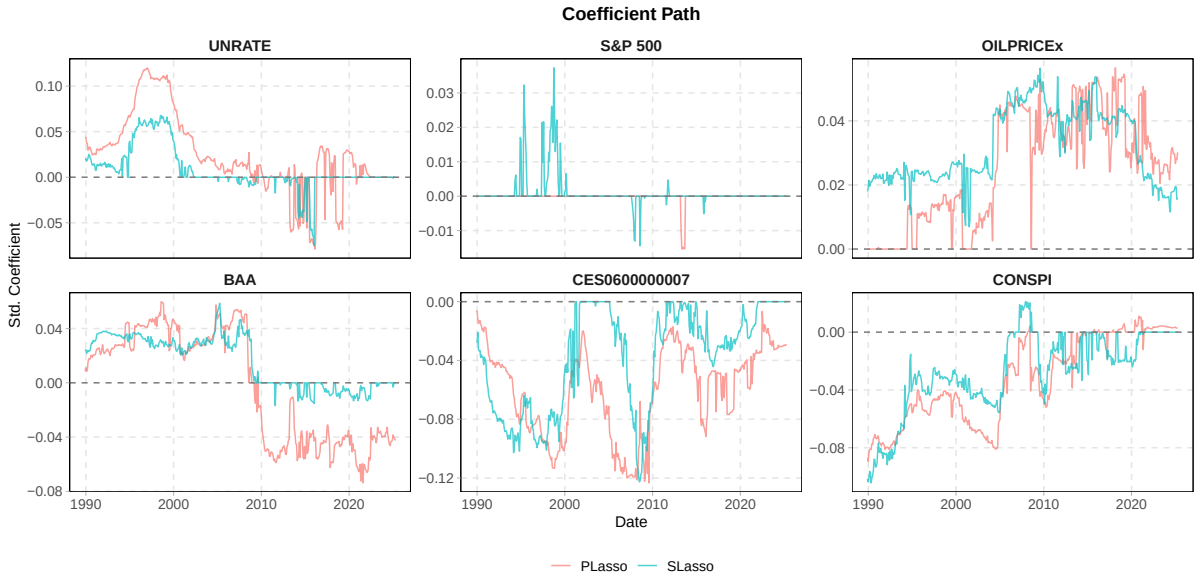
Instead of relying on `top_n`, you can specify exact variables to display via the `variables` argument. Zooming into the financial crisis and pandemic periods shows how the selected predictors and their magnitudes shift during turbulent episodes:

```
p_crisis <- autoplot(bt_results,
  type = "coef_heatmap", methods = "SLasso",
  variables = c("UNRATE", "OILPRICEx", "BAA", "CES0600000007", "CONSPI"),
  date_range = c("2007-01-01", "2010-01-01")
) + ggtitle("Financial Crisis (2007--2010)")

p_pandemic <- autoplot(bt_results,
  type = "coef_heatmap", methods = "SLasso",
  variables = c("UNRATE", "OILPRICEx", "BAA", "CES0600000007", "CONSPI"),
  date_range = c("2019-01-01", "2024-01-01")
) + ggtitle("Pandemic (2019--2024)")

p_crisis + p_pandemic +
  plot_layout(guides = "collect") &
  scale_fill_gradient2(
    low = "#2196F3", mid = "white", high = "#F44336", midpoint = 0,
    limits = c(-0.15, 0.15), oob = scales::squish, n.breaks = 5,
    guide = guide_colorbar(barwidth = 10, barheight = 0.5)
  ) &
  theme(legend.position = "bottom")
```



The `"coef_path"` type shows each variable's coefficient trajectory over time, faceted so that multiple methods can be compared side by side. Like `"coef_heatmap"`, it supports `standardize`, `date_range`, and `variables`:

```
autoplot(bt_results,
  type = "coef_path",
  methods = c("PLasso", "SLasso"),
  variables = c("UNRATE", "S&P 500", "OILPRICEx", "BAA", "CES0600000007", "CONSPI")
)
```



3 Extending the Framework

The sections above illustrate the estimators shipped with the package. This section changes the perspective: `LasForecast` is designed as an *infrastructure* for predictive regression backtesting. The estimation, rolling-forecast, and visualization layers communicate exclusively through a single lightweight S3 class, `lasforecast_model`. Any estimator of the linear predictive regression Equation 1 that returns this class — whatever its internal machinery — plugs into the entire pipeline: rolling and expanding windows, loss metrics and benchmark ratios, and every `autoplot()` type of the previous section. We first document the class and then walk through a concrete extension, best subset selection (BSS), an estimator outside the LASSO family that entered the package in exactly this way.

3.1 The `lasforecast_model` Class

A fitted model is a plain R list carrying the class attribute `"lasforecast_model"`. For instance, the SLasso fit from the estimation section has the structure:

```
str(mod_s, max.level = 1)
```

```
List of 4
 $ fit         :List of 12
  ..- attr(*, "class")= chr [1:2] "elnet" "glmnet"
 $ lambda      : num 0.0102
 $ coefficients: num [1:112] 2.63638 0.00688 0 -2.17906 0 ...
 $ method      : chr "lasso"
 - attr(*, "class")= chr "lasforecast_model"
```

The contract is deliberately minimal — for forecasting purposes, every linear predictive regression estimator reduces to a coefficient vector and a label:

- `coefficients` (required): numeric vector of estimates, intercept first when the model has one. This is the only component the prediction layer relies on: `predict.lasforecast_model()` computes the linear forecast and infers the presence of an intercept from the vector length.
- `method` (required): character string identifying the estimator, used by `print()` and as the label in summaries and plots.
- `lambda` or `tuning_param` (optional): the selected tuning parameter, recorded window by window in rolling exercises.
- `fit` (optional): arbitrary method-specific detail (for SLasso, the underlying `glmnet` object). Nothing downstream depends on it.

At every window, `roll_predict()` — and therefore `backtest()`, which is built on it — interacts with the fitted object only through this contract: it stores the slope coefficients, the tuning parameter, and the number of nonzero slopes, and it forms the h-step-ahead forecast via `predict(model, newx = ...)`. All evaluation and visualization machinery consumes these stored quantities, never the method's internals. Adding a new method to the package therefore takes two steps:

1. Write an estimation function that returns a `lasforecast_model`:

```r
my_method <- function(x, y, ...) {
  # ... estimation producing alpha_hat, beta_hat, and a tuning value ...
  structure(
    list(
      coefficients = c(alpha_hat, beta_hat), # intercept first
      method = "my_method",
      tuning_param = tuning_value, # optional
      fit = NULL # optional method-specific details
    ),
    class = "lasforecast_model"
  )
}
```

2. Register the method name in `roll_predict()`: one entry in its catalogue of valid methods and one line in its per-window `switch()` dispatch.

The generics `print()`, `coef()`, and `predict()` work for the new method out of the box, and so do the `backtest()` summaries and plots.

3.2 Example: Best Subset Selection

The LASSO penalty $\|\theta\|_1$ is the convex relaxation of the cardinality constraint $\|\theta\|_0 = \sum_{j=1}^{p} 1\{\theta_j \neq 0\}$. Best subset selection attacks the nonconvex problem directly:

$$\hat{\theta}^{BSS} = \underset{\theta}{\operatorname{argmin}} \ \|y - W\theta\|_2^2 \quad \text{subject to} \quad \|\theta\|_0 \leq k, \tag{8}$$

the best-fitting regression with at most k active predictors (the intercept is handled by demeaning and not counted in k). Bertsimas et al. (2016) show that modern mixed integer optimization (MIO) solves Equation 8 exactly at practically relevant scales. The package implements their approach in `bss()`: for each candidate subset size, a discrete first-order algorithm supplies a warm start to the Gurobi MIO solver, which returns the exact constrained least squares optimum; the subset size is fixed by the user or selected by information criterion (BIC by default) over a candidate set, by default $k = 0, 1, \ldots, k_{\max}$.

As an estimator, BSS differs from the LASSO family in almost every respect: the optimization is combinatorial rather than convex, the solver is external (Gurobi) rather than `glmnet`, and the tuning parameter is the integer subset size k rather than a continuous penalty level λ. It is also exactly scale-equivariant: `bss()` solves the problem on the raw data with no standardization at any stage, and rescaling a predictor merely rescales its coefficient inversely, leaving the selected subset and the forecasts unchanged — the PLasso–SLasso distinction is moot for BSS. Unlike the LASSO variants above, BSS has no established asymptotic theory under mixed-persistence regressors; within the package it serves as an agnostic ℓ_0 benchmark for the LASSO-family methods. These differences make it a natural stress test of the interface. Since Gurobi is commercial software (free academic licenses are available), the `gurobi` R package is declared in `Suggests`, and the two chunks below evaluate only on machines where it is installed.

```
mod_bss <- bss(fredmd, y = "infl_cpi", k = 5)
mod_bss
```

```
LasForecast Model
  Method: bss
  Coefficients: 112 ( 6 nonzero )
  Tuning param: 5
```

Despite the entirely different machinery, the returned object obeys the same contract — `method`, `tuning_param` (the subset size), and an intercept-first `coefficients` vector — so the generic methods apply unchanged:

```
b_bss <- coef(mod_bss)
names(b_bss) <- c("(Intercept)", setdiff(names(fredmd), c("date", "infl_cpi")))
b_bss[b_bss != 0]
```

```
(Intercept)      AWHMAN     BUSINVx         GS5    TB6SMFFM    CPITRNSL
  3.5736158  -0.0827357  14.8461759   0.2032148  -0.1420707   6.2216908
```

Here we fix `k = 5` to keep the vignette light; with the default `k = NULL`, `bss()` selects the subset size by information criterion and stores the search path in `fit$ic_path`. `predict()` works exactly as in the SLasso example above.

Step 2 of the recipe is the registration inside `roll_predict()`, which in stylized form reads:

```
switch(method,
  "SLasso" = lasso(x_est, y_est, method = "lasso", scale_x = TRUE, ...),
  # ... other built-in methods ...
  "BSS" = bss(x_est, y_est, k = bss_k, predictive = FALSE, ...)
)
```

Note the explicit `predictive = FALSE`: the rolling loop already pairs y_t with W_{t-1} when forming each window sample, so the per-window call must disable the estimator's own predictive alignment. With this entry, `"BSS"` joins the method set of `roll_predict()` and `backtest()`: its `bss_k` argument fixes the per-window subset size, while the default `bss_k = NULL` re-selects k by information criterion within every window, at the cost of $k_{\max}$ MIO solves per window. All loss summaries, benchmark ratios, and `autoplot()` types of the previous section then apply to BSS automatically. The same two steps — return a `lasforecast_model`, register the name — extend the backtesting infrastructure to any estimator of Equation 1.

References

Bertsimas, Dimitris, Angela King, and Rahul Mazumder. 2016. "Best Subset Selection via a Modern Optimization Lens." *The Annals of Statistics* 44 (2): 813–52. https://doi.org/10.1214/15-AOS1388.

Gao, Zhan, Ji Hyung Lee, Ziwei Mei, and Zhentao Shi. 2026. "LASSO Inference for High Dimensional Predictive Regressions." *Journal of Econometrics* 255: 106240. https://doi.org/10.1016/j.jeconom.2026.106240.

Lee, Ji Hyung, Zhentao Shi, and Zhan Gao. 2022. "On LASSO for Predictive Regression." *Journal of Econometrics* 229 (2): 322–49. https://doi.org/10.1016/j.jeconom.2021.02.002.

McCracken, Michael W., and Serena Ng. 2016. "FRED-MD: A Monthly Database for Macroeconomic Research." *Journal of Business & Economic Statistics* 34 (4): 574–89.

Mei, Ziwei, and Zhentao Shi. 2024. "On LASSO for High Dimensional Predictive Regression." *Journal of Econometrics* 242 (2): 105809. https://doi.org/10.1016/j.jeconom.2024.105809.

Phillips, Peter C. B., and Tassos Magdalinos. 2009. "Econometric Inference in the Vicinity of Unity." *CoFie Working Paper*.

Stambaugh, Robert F. 1999. "Predictive Regressions." *Journal of Financial Economics* 54 (3): 375–421.

Tibshirani, Robert. 1996. "Regression Shrinkage and Selection via the LASSO." *Journal of the Royal Statistical Society: Series B* 58 (1): 267–88.

Zhang, Cun-Hui, and Stephanie S. Zhang. 2014. "Confidence Intervals for Low Dimensional Parameters in High Dimensional Linear Models." *Journal of the Royal Statistical Society: Series B* 76 (1): 217–42.

Zou, Hui. 2006. "The Adaptive LASSO and Its Oracle Properties." *Journal of the American Statistical Association* 101 (476): 1418–29.